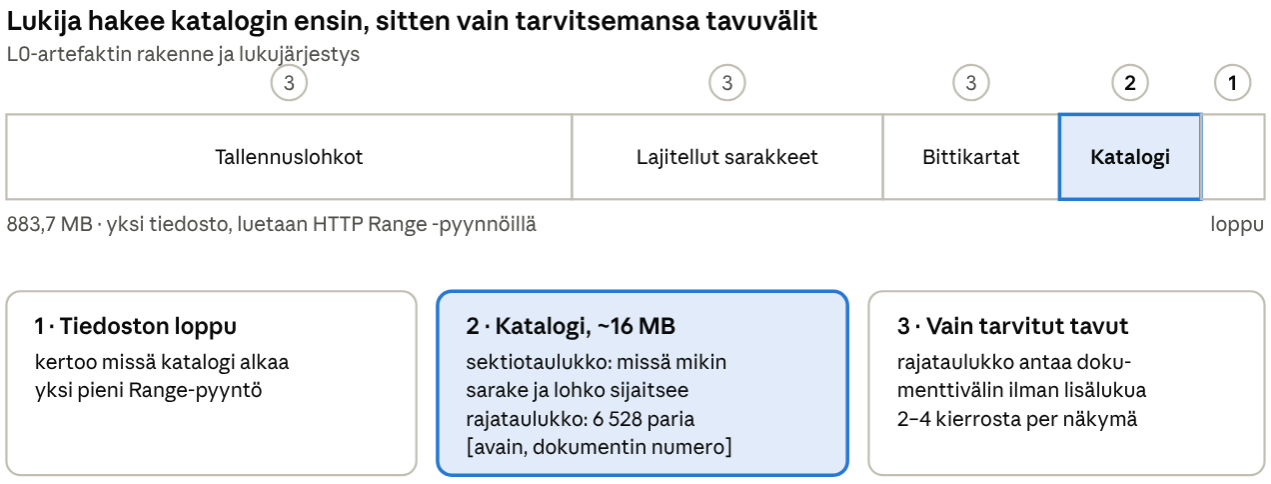
L0-indeksi ja MML:n maastokartta-aineisto

Oct 10, 2026 · @Matti Katila

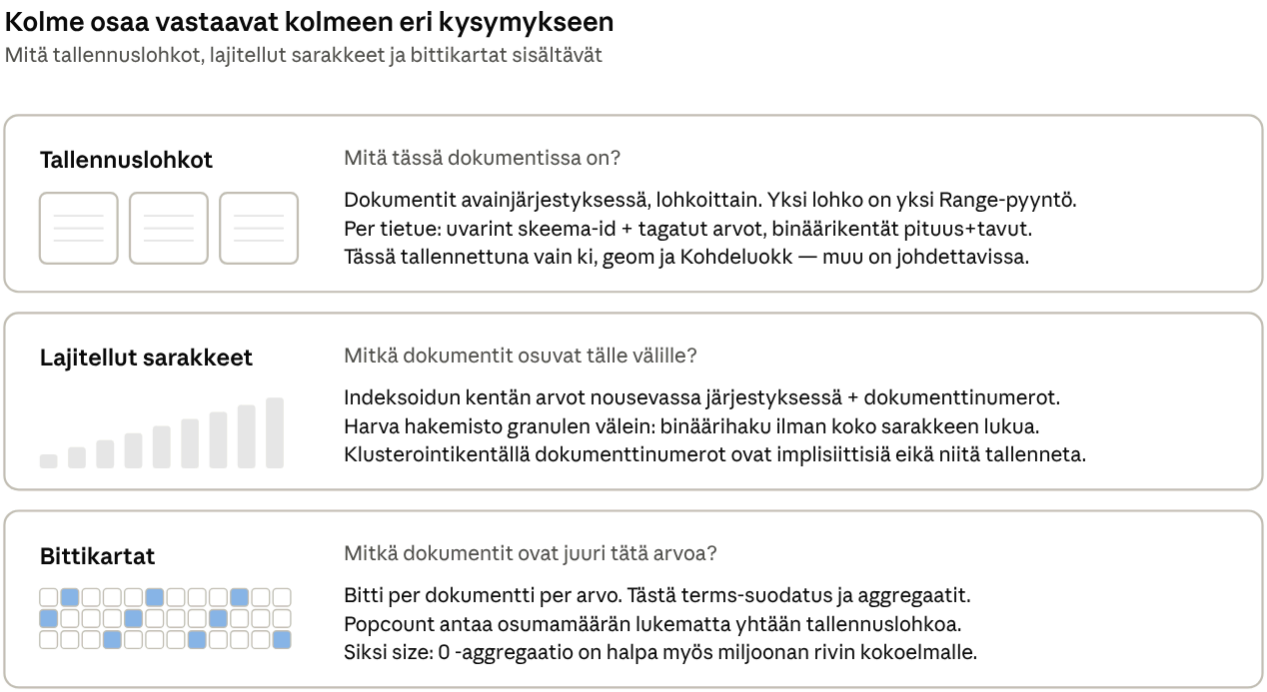
Yleiskuvaus

Maanmittauslaitoksen koko maan maastokartta-aineisto mahtuu yhteen 883 MB:n tiedostoon, jota selain lukee HTTP Range -pyynnöillä ilman palvelinta. Mikä tahansa näkymä maksaa **2–4 kierrosta ja 20–40 ms**, zoomista riippumatta.

L0 on muuttumaton indeksimuoto: dokumentit tallennetaan yhden avaimen mukaiseen järjestykseen, ja tiedoston lopusta alkava katalogi kertoo missä mikäkin sarake, bittikartta ja lohko sijaitsee. Lukija hakee ensin katalogin, sitten tarvitsemansa tavuvälit. Koska dokumentit ovat avainjärjestyksessä, yhtenäinen avainväli on yhtenäinen tavualue — ja se on yksi Range-pyyntö.



L0-artefaktin rakenne · 5 osaa, 3 lukuvaihetta



Artefaktin kolme osaa · mitä kukin sisältää

Jako on työnjako: bittikartat kertovat **mitkä** dokumentit täsmäävät, lajitellut sarakkeet **mille välille** ne osuvat, ja vasta tallennuslohkot sisältävät itse sisällön. Kaksi ensimmäistä ovat halpoja, kolmas maksaa — ja siksi koko työ on sen minimoimista, kuinka monta lohkoa joudutaan lukemaan.

Lukujärjestys on lopusta alkuun: tiedoston viimeiset tavut kertovat missä katalogi on, katalogi kertoo missä kaikki muu on, ja vasta kolmas pyyntö hakee dataa.

Aineisto on MML:n maastokartta-aineisto vektorimuodossa, mittakaavoissa 1:100 000 ja 1:250 000, Shapefile-muodossa ladattuna. Koordinaatisto on EPSG:3067 (ETRS-TM35FIN), yksikkö metri.

Artefakti	883,7 MB
Kohteita	6 683 615
Kokoelmia	72 (36 tasoa × 2 mittakaavaa)
Kierroksia per näkymä	2–4
Aika per näkymä	20–40 ms
Tavuja per näkymä	10–45 MB tyylistä riippuen

Työn päälle rakennettiin selainkäyttöinen karttasovellus, jossa piirtosäännöt annetaan JSON-tyylitiedostona. Sillä tehtiin yksitoista karttaa: peruskartta, korkeusvyöhykkeet, luonnonsuojelualueet, viisi paikannimien levinneisyyskarttaa, oja- ja tiepintakartta.

Tämä raportti kuvaa indeksointiavaimen, aineistolle tehdyt muunnokset, mittaamalla löytyneet erityispiirteet, esimerkkihakuja ja sen mitä toisen aineiston käyttöönotto vaatii.

Indeksointiavain ja Hilbert-käyrä

Katalogissa on avaimesta vain **metatiedot, ei arvoja**. Itse avaimet ovat tiedoston rungossa lajiteltuna sarakkeena, ja katalogi kertoo missä se sarake alkaa ja miten siihen hypätään.

Katalogi sisältää klusteroinnista nämä: kentän nimi ja suunta (`avain`, nouseva), **rajataulukko** eli `[avain, dokumentin numero]` -pari joka granulen välein, sekä lippu `implicitDocs`. Tässä artefaktissa granule on 1024 ja rajataulukossa on 6 528 merkintää.

Rajataulukko on se joka tekee hausta halvan. Kun kysely antaa avainvälin, lukija etsii taulukosta välin päätä ympäröivät merkinnät ja saa suoraan dokumenttinumeroiden välin — **ilman yhtään lisälukua tiedostosta**. Vasta sen jälkeen haetaan ne tallennuslohkot jotka osuvat väliin.

`implicitDocs` tarkoittaa että kun klusterointikentässä ei ole aukkoja, **dokumentin numero on sama kuin sen sijainti lajitellussa sarakkeessa**. Silloin erillistä dokumenttinumeroiden taulukkoa ei tarvitse tallentaa lainkaan.

Katalogi luetaan kerran avattaessa. Tässä artefaktissa se on noin 16 MB, ja sen jälkeen jokainen kysely on puhdasta tavuvälien hakua.

Koko muodon lupaus nojaa siihen, että avaimia on vain yksi. Kartta tarvitsee kaksi asiaa peräkkäisyydeltä: *mikä taso* ja *missä päin Suomea*. Molemmat mahtuvat yhteen lukuun.

```
\text{avain} = \text{kokoelma} \times 4^{18} + \text{hilbert}
(\text{keskipiste})
```

Koska 4^18 = 68 719 476 736 = 2^36, kertolasku on bittisiirto: `(kokoelma << 36) | hilbert`. Ylemmät bitit ovat kokoelma, alemmat 36 sijainti.

Kokoelma on avaimen etuliite

Kokoelma varaa itselleen 68,7 miljardin avaimen lohkon, jonka sisällä sijainti ei voi vuotaa naapurikokoelmaan. Tästä seuraa, että **yksi kokoelma on aina yhtenäinen tavualue** — ja siihen perustuu luvun "Erityishavainnot" tärkein tulos.

72 kokoelmaa × 68,7 mrd = 4,95 biljoonaa, ja `Number.MAX_SAFE_INTEGER` on 9,01 biljardia. Avain mahtuu tavalliseen JavaScript-lukuun ilman BigIntiä, ja juuri siksi `ORDER` on 18 eikä enemmän.

Hilbert-käyrä: kaksi ulottuvuutta yhdeksi luvuksi

`ORDER = 18` tarkoittaa 262 144 × 262 144 -ruudukkoa Suomen laatikon päälle (x 50 000–760 000, y 6 570 000–7 800 000). Solu on **2,71 m × 4,69 m**. Hilbert-käyrä numeroi solut nollasta 68 719 476 735:een niin, että käyrällä peräkkäiset numerot ovat myös maastossa vierekkäin.

Mitattuna yhdestä pisteestä Keski-Suomessa:

siirtymä maastossa	ero avaimessa
10 m	15
100 m	11 768
1 000 m	146 021
10 000 m	13 878 186

Hilbert eikä Z-order (Morton) siksi, että Hilbert-käyrä ei tee pitkiä hyppyjä: peräkkäiset indeksit ovat aina naapurisoluja. Z-orderissa käyrä loikkaa ruudukon laidasta toiseen aina kun korkeampi bitti vaihtuu, jolloin sama laatikko hajoaa moninkertaiseen määrään välejä.

Laatikosta väleiksi

Näytön suorakaide hajotetaan Hilbert-väleiksi laskeutumalla nelipuussa: laatikon sisällä kokonaan oleva solu on yksi väli, ulkopuolinen pudotetaan, rajalla oleva pilkotaan kunnes budjetti loppuu. Hajotelma ei lue yhtään tavua — se on pelkkää geometriaa, joten välit tiedetään ennen ensimmäistä hakua.

Mitattu 1500 × 950 pikselin näytöllä, budjetti 64 väliä:

m/px	välejä	ylihaku
40	11	1,29×
140	5	1,22×
300	6	1,17×
600	3	1,08×

Budjetin nosto 1024:ään vie ylihaun 1,17 → 1,07 kertaiseksi 300 m/px:llä. Se on tyylikohtainen asetus, koska lisävälit maksavat hieman kyselyn koossa.

Miten aineistoa piti muokata

MML:n aineisto tulee Shapefile-paketteina karttalehdittäin: `.shp` geometria, `.dbf` attribuutit, `.cpg` merkistö. Lukijat kirjoitettiin itse, ilman riippuvuuksia — yhteensä 89 riviä.

Kokoelmanimi: taso ja mittakaava yhdessä

Kokoelma on `taso@mittakaava`, esimerkiksi `TieViiva@100k`. Mittakaava kuuluu nimeen eikä pelkäksi kentäksi, koska **yleistetty mittakaava ei ole tarkemman osajoukko**: sen kohteet ovat eri kohteita, kartografin yhdistämiä ja siirtämiä. Erillisinä kokoelmina kumpikin saa oman yhtenäisen lohkonsa, ja näkymä lukee vain sen mittakaavan jota käyttää.

Dokumentin rakenne

```
kenttä,sisältö,indeksoitu,tallennettu
kokoelma,taso@mittakaava,kyllä,ei
taso,layer-nimi,kyllä,ei
mittakaava,100k tai 250k,kyllä,ei
Kohdeluokk,MML:n kohdeluokka,kyllä,kyllä
x / y,keskipiste metreinä,kyllä,ei
avain,klusterointiavain,kyllä (klusterointi),implisiittinen
ki,kokoelman indeksi numerona,ei,kyllä
geom,varint-pakattu geometria,ei,kyllä
```

Mitä jätettiin tallentamatta

Ensimmäinen mittaus paljasti, että **65 % attribuuttitavuista oli kenttiä jotka olin itse lisännyt** (`kokoelma`, `avain`, `id`, `x`, `y`) — kaikki johdettavissa. Sijainti on jo geometriassa, kokoelma avaimen korkeissa biteissä, dokumentin id on sen positio. Erillinen `storedFields` -sallilista pudotti yhden karttalehden artefaktin 3,69 → 2,87 MB.

Poikkeus on `ki`: kokoelman indeksi **tallennetaan** numerona. Nimi on johdettavissa avaimen korkeista biteistä, mutta vain granulen tarkkuudella ilman lisähakua, ja piirtäjän on tiedettävä täsmälleen mihin tasoon kohde kuuluu. Kaksi varint-tavua toistuvaa merkkijonoa vastaan on halpa tapa olla tarkka.

Geometrian pakkaus

Geometria kvantisoidaan metrin ruudukkoon ja koodataan zigzag-varint-erotuksina. Mitattu 389 023 pisteellä: häviö enintään 0,500 m, pakkaussuhde 5,7×, **2,58–2,83 tavua per piste**.

Pikkuasiat jotka olisivat rikkoneet aineiston

- MML:n puuttuvan arvon sentineli on `-29999`. Se muunnetaan `null`iksi ingestissä, muuten se olisi tullut mukaan lukuarvona aggregaatteihin.
- Tyhjä merkkijono muunnetaan niin ikään `null`iksi.
- Merkistö luetaan `.cpg`-tiedostosta, ei oleteta.
- Dokumentit on **lajiteltava avaimen mukaan ennen rakennusta**; L0 ei lajittele niitä puolestasi.

Skeemataulukko

Kartta-aineistossa eri tasoilla on eri kentät, ja harva tietue sisältää niitä kaikkia. Rakentaja kerää havaitut kenttien yhdistelmät skeemataulukoksi ja kirjoittaa per tietue vain `uvarint skeemaId` + tagatut arvot. Jos erilaisia yhdistelmiä on yli 1024, se putoaa takaisin yhteen unioniskeemaan.

Erityishavainnot

1. Suodatin valitsee rivit, ei tavuja

Tämä on työn tärkein tulos. Suodatus `TieViiva@250k`-kokoelman 458 161 rivistä 12 205 valtatiehen — 2,7 % riveistä — säästi **3,6 % tavuista**.

kysely	osumia	siirtoa
kaikki tiet	458 161	16,78 MB
vain valtatiet	12 205	16,17 MB

Syy: 12 205 valtatietä on ripoteltuna jokaiseen tallennuslohkoon. Valikoivuus ei muutu I/O:ksi, koska klusterointiavain ei tiedä luokasta mitään.

Kyllä, ja tämä on muodon todellinen rajoite: **vain yksi järjestys voi olla halpa kerrallaan.** Teitä haetaan tässä työssä aina sijainnin mukaan, joten ne on lajiteltu sijainnin mukaan. Jos niitä tarvittaisiin esimerkiksi tienumeron tai päällysteen mukaan, se haku lukisi koko kokoelman.

Mitattu esimerkki samasta aineistosta: `Paallyste`-kenttä ei ole indeksoitu, joten koko maan sora/piki-jaottelu vaatii koko `TieViiva`-kokoelman lukemisen — 43,4 MB. Sijainnin mukaan rajattu sama kysely maksaa 19,1 MB pienemmällä alueella.

Vaihtoehtoja on neljä, ja valinta riippuu siitä kuinka usein toinen järjestys tarvitaan.

keino	kustannus	milloin
Lue koko kokoelma ja suodata selaimessa	kokoelman koko kerran	harvoin tarvittava näkymä
Vie attribuutti avaimen etuliitteeseen	ei lisäkustannusta, mutta sijaintijärjestys menee toissijaiseksi	kun se attribuutti on **ensisijainen** rajaus
Luo attribuutista oma kokoelma (tiering)	artefakti kasvaa hieman, molemmat järjestykset halpoja	kun molempia tarvitaan usein
Rakenna toinen artefakti toisella avaimella	kaksinkertainen levytila	kaksi aidosti eri käyttötarkoitusta

Käytännössä kolmas on yleensä oikea. Koska kokoelma on avaimen etuliite, `TieViiva@100k#paallystetty` ja `TieViiva@100k#sora` olisivat kumpikin yhtenäisiä lohkoja, joiden **sisällä sijaintijärjestys silti säilyy**. Yksi attribuutti saadaan siis avaimeen ilman että sijainti menetetään — mutta vain yksi, ja valinta on tehtävä ennen ajoa.

Muodossa ei ole toissijaisia indeksejä siinä mielessä kuin tietokannassa. Bittikartat kertovat *mitkä* dokumentit täsmäävät halvalla, mutta niiden *lukeminen* maksaa aina sen mukaan kuinka hajallaan ne ovat.

Korjaus on viedä valitsin avaimeen, ei kyselyyn. Koska kokoelma on avaimen etuliite, luokan vieminen kokoelman nimeen tekee siitä yhtenäisen. Rakensin todisteen samasta datasta uudelleen, kokoelmat `TieViiva@250k#a` (tieluokka) ja `KarttanimiPiste@250k#1..5` (mittakaavarelevanssi):

kerros	nykyinen	avaimessa	säästö
Valtatiet	19,84 MB	0,50 MB	**40×**
Nimistö, ylin taso	10,02 MB	0,04 MB	**227×**

Koko maan näkymä putoaisi ~30 MB:stä ~1 MB:hen. Muutos on ingestissä yhden merkkijonon mittainen, mutta vaatii uuden ajon.

2. Indeksi tuntee kohteen vain keskipisteestään

Avain on `hilbert(keskipiste)`, joten 32 pisteen polygoni puristuu yhdeksi pisteeksi. Kohde löytyy vain jos sen *painopiste* osuu laatikkoon — ei jos sen geometria leikkaa sen. Iso järvi katoaa sillä hetkellä kun sen keskikohta valuu ruudun ulkopuolelle.

Mitattu vertaamalla geometriaa näkymään:

näkymä	VesiAlue hukassa	HallintoalueRaja	TaajamaAlue
koko maa	0,0 %	0,0 %	0,0 %
maakunta	0,6 %	10,8 %	1,8 %
kaupunki	2,2 %	**29,4 %**	8,5 %
kortteli	**10,0 %**	0,0 %	**36,4 %**

Hukka kasvaa zoomatessa, ei loitonnettaessa. Prosentti myös aliarvioi vaikutuksen: kymmenestä järvestä yksi on 10 %, mutta se yksi peittää puolet ruudusta.

Kiertotapa on leventää hakualuetta kerroskohtaisesti kohteen tyypillisen koon verran. Mitattu panoroimalla 13 ruutua Päijänteen yli: 25 km:n lattia antaa 100 % saannon joka zoomilla ja maksaa syvimmillään 0,16 → 0,56 MB. Se on heuristiikka, ei takuu; oikea korjaus on kokoluokittelu ingestissä, jolloin kunkin luokan suurin säde tiedetään.

3. Yleistetty mittakaava ei sisällä kaikkia luokkia

`VesiViiva@250k` ei sisällä luokkaa 36311 (virtavesi alle 2 m) **lainkaan** — 1,59 miljoonaa kohdetta puuttuu kokonaan. Yleistys ei ole harvennus vaan valinta. Jos näkymä tarvitsee luokan, mittakaava on lukittava siihen jossa se on.

4. Attribuutit painavat enemmän kuin geometria

Oletin geometrian hallitsevan karttadataa. Mitattuna **attribuutit 69 %, geometria 31 %**, ja kenttien nimet 77 % attribuuttitavuista. Tästä seurasi sekä positionaalinen koodaus että `storedFields`-sallilista.

5. where vs. having: kerros per kysely maksaa kerroksen verran

Luonnonsuojelutyylin ensimmäinen versio maksoi **73,6 MB**, koska viisi kerrosta luki samaa `SuojeluAlue`-kokoelmaa, kukin omalla `where`-suodattimellaan.

kerros	osumia	tavuja	per kohde
Kansallispuisto	676	6,66 MB	**9,8 kB**

kerros	osumia	tavuja	per kohde
Luonnonpuisto	24	2,20 MB	92 kB
Muu suojelualue	7 929	8,73 MB	1,1 kB

Korjaus: luokkasuodatus `where`stä `having`iin eli selainpuolelle, ja samanlaisten lausekkeiden poisto kyselystä. **73,6 → 23,1 MB**, lausekkeita 6.

Sääntö: `where` sopii kun suodatin pienentää luettavaa aluetta (klusterointiavain, kokoelma). `having` sopii kun kerrokset jakavat saman kokoelman — yksi haku, jako selaimessa. Huomaa kuitenkin että se siirtää työn eikä poista sitä: tavuja säästyy, suoritusaikaa ei. Mitattu hinta on luvussa “Mitä asiakaspuolen suodatus maksaa”.

Hakulausekkeita

Suodatustasoja on kaksi, ja ne on helppo sekoittaa toisiinsa. **Indeksin kyselykieli** on Elasticsearchin osajoukko ja kulkee verkon yli. **Karttatyylin `having` ei ole osa sitä lainkaan** — se on selaimessa ajettava suodatin, joka toimii vasta kun tavut ovat jo tulleet.

Kaksi suodatustasoa

	missä ajetaan	mitä osaa	mitä säästää
indeksin kysely	palvelimella, ennen tavuja	`bool` / `should` / `filter`, `terms`, `range`, `aggs`, `size`	lukualue
tyylin `where`	käännetään yllä olevaksi kyselyksi	`terms` indeksoidulle kentälle	tavuja, jos suodatin osuu klusterointiin
tyylin `having`	selaimessa, haun jälkeen	`in`, `endsWith`, `startsWith`, `gte`, `lte`, `gt`, `lt`	vain mustetta

`endsWith` ja `startsWith` eivät vastaa mitään indeksin kyselyssä. Niitä ei voi viedä sinne, koska muodossa on termihaku eikä sanahakua — ja juuri siksi paikannimien levinneisyyskartat (”mitkä nimet päättyvät sanaan *vuoma*”) hakevat koko kokoelman ja karsivat selaimessa.

Mitä asiakaspuolen suodatus maksaa

Mitattu nyppyläkartan kerroksilla, 170 519 nimeä, koodi poimittuna sanatarkasti sovelluksesta:

	aika	per tietue
raaka `endsWith('tunturi')`	**1,1 ms**	6 ns
yksi kerros `matches()`:n kautta	43,1 ms	253 ns
koko suodatus, 8 kerrosta	**96,0 ms**	563 ns

Vertailu itsessään on ilmainen: 151 miljoonaa `endsWith`-vertailua sekunnissa, koska V8 vertaa lopusta alkaen eikä skannaa merkkijonoa. Kustannus on kääreessä: `Object.entries(L.having)` rakennetaan uudelleen jokaiselle tietueelle jokaisessa kerroksessa, `String(v).toLowerCase()` luo uuden merkkijonon joka kerta, ja `findIndex` käy kerrokset järjestyksessä läpi.

96 ms on **enemmän kuin itse haku**: saman näkymän 45,9 MB tulee verkosta 20 ms:ssä. Esikäännetty suodatin (`Object.entries` kerran per kerros, pienaakkostus kerran per tietue) vie sen 49 ms:ään, mutta oikea korjaus on sama kuin kaikkialla muuallakin tässä työssä: jos valitsin olisi avaimessa, haettavia olisi 11 979 eikä 170 519, ja suodatus putoaisi samassa suhteessa ilman että koodiin kosketaan.

Esimerkkejä indeksin kyselykielestä

Alla todellisia lausekkeita tästä työstä, mitattuine kustannuksineen.

Näkymähaku: bbox Hilbert-välein

Tämä on kartan perushaku. Laatikko hajotetaan väleiksi asiakaspuolella, ja jokainen väli siirretään kokoelman lohkoon lisäämällä `ci * SPAN`.

```
{
  "query": { "bool": { "should": [
    { "range": { "avain": { "gte": 4122669875200, "lte": 4122670923775 } } },
    { "range": { "avain": { "gte": 4122672496640, "lte": 4122673545215 } } }
  ] } },
  "size": 24000
}
```

Mitattu: 5–8 väliä riittää mille tahansa näytölle, ylihaku 1,2–1,5×.

Mistä kyselyn luvut tulevat

Yllä olevan kyselyn `4122669875200` ei ole mielivaltainen — se on `avain`-kentän arvo, eli sama kaava kuin luvussa "Indeksointiavain". **Kysely ei tunne Hilbert-käyrää lainkaan**: se on tavallinen lukuvälihaku. Koko geometria tehdään asiakaspuolella ennen kyselyn

muodostamista.

Vaiheet yhdelle oikealle kohteelle, mitattuna tästä artefaktista:

```
vaihe,arvo
1. geometria,32 pistettä
2. keskipiste (pisteiden keskiarvo),"432 824, 6 840 486 m (ETRS-TM35FIN)"
3. ruudukkosolu (2.71 m x 4.69 m),"141 345, 57 647"
4. hilbert(solu),61 512 288 342
5. kokoelman indeksi (VesiAlue@100k),62
6. AVAIN = 62 x 68 719 476 736 + 61 512 288 342,4 322 119 845 974
```

Näkymähaussa sama tehdään toisin päin. Näytön suorakaide projisoidaan ruudukkoon, hajotetaan Hilbert-väleiksi `[lo, hi]`, ja jokainen väli siirretään kokoelman lohkoon:

```
\text{gte} = ci \times 4^{18} + lo \qquad \text{lte} = ci \times 4^{18} + hi
```

Koska kokoelma on avaimen korkeat bitit, siirto `ci * SPAN` ei voi vuotaa naapurikokoelmaan: `lo` ja `hi` ovat aina alle 4^18.

Kaksi seurausta kannattaa huomata. **Lukija ei tee geometriaa** — se näkee pelkän lukuvälin, joten mikä tahansa muu kaksiulotteinen käyrä kelpaisi muuttamatta lukijaa. Ja **välit tiedetään ennen ensimmäistä tavua**, joten koko näkymä voidaan hakea yhtenä aaltona eikä ketjuna.

Luokkasuodatus välin sisällä

Attribuuttisuodatin ratsastaa oman välinsä mukana, jolloin jokainen piirtosääntö säilyttää oman valintansa ja kaikki on yhä yksi aalto.

```
{
  "query": { "bool": { "should": [
    { "bool": { "filter": [
      { "range": { "avain": { "gte": 4122669875200, "lte": 4122670923775 } }
},
      { "terms": { "Kohdeluokk": ["12111", "12112"] } }
    ] } }
  ] } },
  "size": 24000
}
```

Huomaa luvun "Erityishavainnot" kohta 1: tämä *valitsee* oikein muttei vältämättä säästä tavuja.

Koko kokoelma

```
{
  "query": { "bool": { "filter": [
    { "terms": { "kokoelma": ["VesiViiva@100k"] } },
    { "terms": { "Kohdeluokk": ["36311"] } }
  ] } },
  "size": 2000000
}
```

Mitattu: 1 592 062 kohdetta, **85,2 MB, 1 kierros, 510 ms**.

Aggregaatio: mitä luokkia kokoelmassa on

```
{
  "query": { "bool": { "filter": [ { "terms": { "kokoelma": ["TieViiva@100k"]
} } ] } },
  "size": 0,
  "aggs": { "k": { "terms": { "field": "Kohdeluokk", "size": 40 } } }
}
```

Tämä on halpa: `size: 0` ei hae yhtään tallennettua kenttää, ja termiaggregaatti lasketaan bittikartoista. Juuri tällä selvitettiin että teitä on 1 001 385 ja luokkia yhdeksän.

Suorakaidehaku indeksoidulla koordinaatilla

Kun Hilbert-välejä ei tarvita — esimerkiksi analyysissä — `x` ja `y` ovat indeksoituja ja käytettävissä suoraan.

```
{
  "query": { "bool": { "filter": [
    { "terms": { "kokoelma": ["KarttanimiPiste@100k"] } },
    { "range": { "x": { "gte": 500000, "lte": 600000 } } },
    { "range": { "y": { "gte": 7680000, "lte": 7780000 } } }
  ] } },
  "size": 50000
}
```

Tällä selvisi että Kaldoaivin alueen 2 898 nimestä enemmistö on saamenkielisiä: sme 912, fin 813, smn 666, sms 507.

Mittarit

Lukija raportoi jokaisesta hausta kierrokset, tavut ja ajan. Niitä kannattaa lukea koko ajan — tässä työssä ne paljastivat jokaisen yllä kuvatun ongelman.

Soveltaminen muuhun aineistoon

Mikään tässä ei ole MML-kohtaista. Muoto vaatii aineistolta kolme asiaa.

Aineisto on muuttumaton. L0 rakennetaan kerran ja luetaan sen jälkeen vain. Päivitys tarkoittaa uutta artefaktia. Tämä sopii julkaisuaineistoihin (kartta-aineisto, rekisterikopio, lokiarkisto), ei elävään tietokantaan.

Kyselyillä on yksi hallitseva muoto. Muoto optimoi *yhden* järjestyksen. Jos kyselyt tulevat kahdesta eri suunnasta yhtä usein, toinen niistä on aina hidas.

Dokumentit voidaan lajitella etukäteen. Rakentaja ei lajittele; syöte on annettava avainjärjestyksessä.

Klusterointiavaimen valinta

Tämä on ainoa oikeasti vaikea päätös, ja se kannattaa tehdä ennen ensimmäistä ajoa. Kaava on aina sama muoto:

```
\text{avain} = \text{karkea ryhmä} \times 2^{k} + \text{hieno järjestys}
```

Karkea ryhmä on se jonka mukaan kyselyt ensin rajaavat — kartassa taso, lokissa palvelu, rekisterissä vuosi. Hieno järjestys on se jonka mukaan ne sitten rajaavat: paikka, aika, asiakas.

Esimerkkejä muista aineistoista:

aineisto	karkea ryhmä	hieno järjestys
ajoneuvorekisteri	merkki tai käyttövoima	käyttöönottopäivä
sovelluslokit	palvelu + taso	aikaleima
kiinteistörekisteri	kunta	sijainti (Hilbert)
viestiarkisto	kanava	aikaleima

Kaksiulotteiselle datalle Hilbert; yksiulotteiselle riittää arvo sellaisenaan.

Tarkistuslista

1. **Valitse klusterointiavain ensin.** Sen muuttaminen jälkikäteen on täysi uudelleenrakennus.
2. **Vie jokainen usein käytetty valitsin avaimen etuliitteeseen**, älä pelkkään kyselyyn. Tämän työn mitattu ero oli 40–227-kertainen.

3. **Indeksoi vain se mitä suodatetaan** (`indexFields`) ja **tallenna vain se mitä näytetään** (`storedFields`). Johdettavat kentät pois: 65 % attribuuttitavuista oli sellaisia.
4. **Mittaa, älä arvaa.** Lukija raportoi kierrokset ja tavut; lue ne jokaisesta kyselystä.
5. **Jos kohteella on laajuutta** (polygoni, aikaväli), muista että avain tuntee vain yhden pisteen siitä. Joko levengä hakua tai luokittele kohteet koon mukaan.
6. **Palvele tiedosto Range-kelpoisella palvelimella.** `python3 -m http.server` vastaa Range-pyyntöön 200:lla, jolloin lukija lukisi vääriä tavuja; se hylkää sellaisen vastauksen.

Milloin tämä ei sovi

- Dataa päivitetään jatkuvasti.
- Kyselyt ovat vapaamuotoisia eikä mitään järjestystä voi suosia.
- Tarvitaan liitoksia useiden kokoelmien välillä.
- Tarvitaan tekstihakua; muodossa on termihaku, ei sanahaku.

Avoimet työt tässä työssä

työ	vaikutus	vaatii
Kokoelmien tiering luokan mukaan	40–227× hallitsevaan kustannukseen	uuden ingestin
Kokoluokittelu isoille kohteille	poistaa keskipisteen sokean pisteen	uuden ingestin
Avainsarakkeen luku ilman geometriaa	ojakartta 146 → ~75 MB	kaksi lisäystä lukijaan